FINAL PROJECT REPORT

Campus Navigation Assistant using A*

Artificial Intelligence | Review 3

Team: Campus Navigation

AI syllabus module: Intelligent Search Techniques

Topic: Heuristic Graph Search using A*

Team members
Rohit Arya (24BCE0819)
Nayan Gattani (24BDS0117)
Aditya Kumar (24BCE0205)
Ashish Kumar (24BKT0059)

VIT Vellore

Submitted for the Artificial Intelligence course project

Abstract

This project implements a campus pedestrian navigation assistant for VIT Vellore. The campus is represented as a weighted graph extracted from OpenStreetMap (OSM), where walkway intersections are nodes and walkable segments are weighted by physical distance. A* search uses an admissible straight-line heuristic to find a minimum-cost route efficiently. Dijkstra's algorithm provides an optimal weighted-search reference, while Breadth-First Search (BFS) demonstrates the difference between minimizing graph hops and minimizing physical distance.

The implementation includes a Python command-line demonstration, a local web service, and a Leaflet-based browser interface. On the representative Main Gate UNI Entrance to A Block route, A* and Dijkstra return the same 1,501.9 m physical route distance, while A* expands 413 nodes compared with Dijkstra's 1,004. BFS returns a 1,570.5 m route because it optimizes the number of segments rather than their lengths.

The application also supports a user-selected crowd scenario. Normal, moderate, and high settings apply 1.00x, 1.25x, and 1.50x multipliers to walkway costs before re-running all three algorithms. This demonstrates dynamic cost modeling while clearly distinguishing a scenario setting from live crowd sensing.

Keywords: A*, Dijkstra, BFS, heuristic search, graph search, campus navigation, OpenStreetMap

1. Introduction and Problem Statement

Navigating a large university campus can be time-consuming, particularly when students must travel between distant academic blocks during short class-transition windows. A visually direct route is not necessarily a walkable route because campus paths, junctions, gates, and access roads constrain movement.

The project models campus navigation as an Artificial Intelligence search problem. Given a source landmark and a destination landmark, the system returns an estimated walking route, distance, time, and intermediate checkpoints. The problem is relevant to the AI syllabus because it applies state-space representation, graph search, heuristic design, path-cost evaluation, optimality, and empirical algorithm comparison.

Scope and boundaries: The implementation covers static pedestrian routing on the supplied OSM-derived graph. It does not claim live traffic prediction, official building-entrance accuracy, indoor navigation, accessibility-aware routing, or personalized mobility modeling.

2. Objectives

• Represent the VIT Vellore pedestrian network as a weighted graph.

• Implement A* search with an admissible Euclidean heuristic.

• Implement Dijkstra and BFS as comparison algorithms.

• Provide both command-line and browser-based demonstrations.

• Compare route cost, nodes expanded, and measured search time.

• Validate correctness through automated tests and explicit input checks.

3. AI Formulation

AI component	Project representation
States (S)	Walkway intersections, pathway junctions, and snapped landmark nodes.
Initial state	The graph node nearest to the selected source landmark.
Actions	Move along an adjacent traversable walkway edge.
Transition model	Return the neighboring node reached by traversing an edge.
Goal test	Check whether the current node is the destination node.
Path cost g(n)	Accumulated physical distance in metres, including endpoint snap offsets.
Heuristic h(n)	Straight-line Euclidean distance from the node to the goal.
Evaluation f(n)	A*: f(n) = g(n) + h(n). Dijkstra uses h(n) = 0.

4. Algorithm and Implementation

The team implemented the search core in campus_nav.py using Python's standard library. No external pathfinding library or pretrained model supplies the route. The browser interface uses Leaflet for map display, while the local web service calls the same search functions as the command-line demonstration.

A* pseudocode

```
A_STAR(graph, start, goal):
1. frontier := priority queue containing (h(start), start)
2. g[start] := 0; parent[start] := empty
3. while frontier is not empty:
   current := remove node with minimum g + h
   if current = goal: reconstruct parent links and return route
   for each neighbor of current: update g, parent and priority if improved
4. return no route if the frontier becomes empty
```

Key data structures: adjacency lists represent the weighted graph; a binary heap implements the frontier; dictionaries store best-known costs and parent pointers; and landmark metadata stores names, categories, and endpoint snap offsets.

5. Dataset and System Architecture

The included campus_map.json contains 1,965 mapped walkway/access nodes and 2,131 walkway edges, with named campus places and hostels. Edge weights are calculated from OSM geometry using great-circle distance. Named landmarks are snapped to nearby walkway nodes; the straight-line snap offsets are reported separately in the UI.

The system has three cooperating layers: (1) the browser interface in index.html, which displays controls, map tiles, route layers, and comparison cards; (2) the Python web service in web_app.py, which exposes /api/map and /api/route; and (3) the shared search core in campus_nav.py, which is also used by the CLI.

6. Algorithms Implemented

A*: A priority queue ranks frontier nodes by f(n) = g(n) + h(n). The straight-line heuristic is checked against every loaded edge so it cannot overestimate the cost of traversing that edge. This makes the heuristic admissible and consistent for the supplied graph.

Dijkstra: Uses the same weighted graph and priority-queue structure but sets h(n) to zero. It is the optimal-cost reference for the experiment.

BFS: Uses a FIFO queue and minimizes the number of graph edges. It ignores distance weights during search, so its physical route may be longer when segment lengths differ.

7. Implementation Workflow

• Load and validate the JSON graph, node coordinates, landmarks, and edges.

• Resolve human-readable landmark names to their nearest mapped graph nodes.

• Initialize the selected search algorithm and expand candidate nodes.

• Store parent pointers and reconstruct the route when the goal is reached.

• Add endpoint snap offsets, estimate walking time at 1.4 m/s, and present the route with its metrics.

8. Live Demonstration Evidence

The screenshot was captured from the running local web application for the default Main Gate UNI Entrance to A Block route under the high crowd scenario. It shows the campus map, route summary, selected crowd setting, and comparison cards for A*, Dijkstra, and BFS.

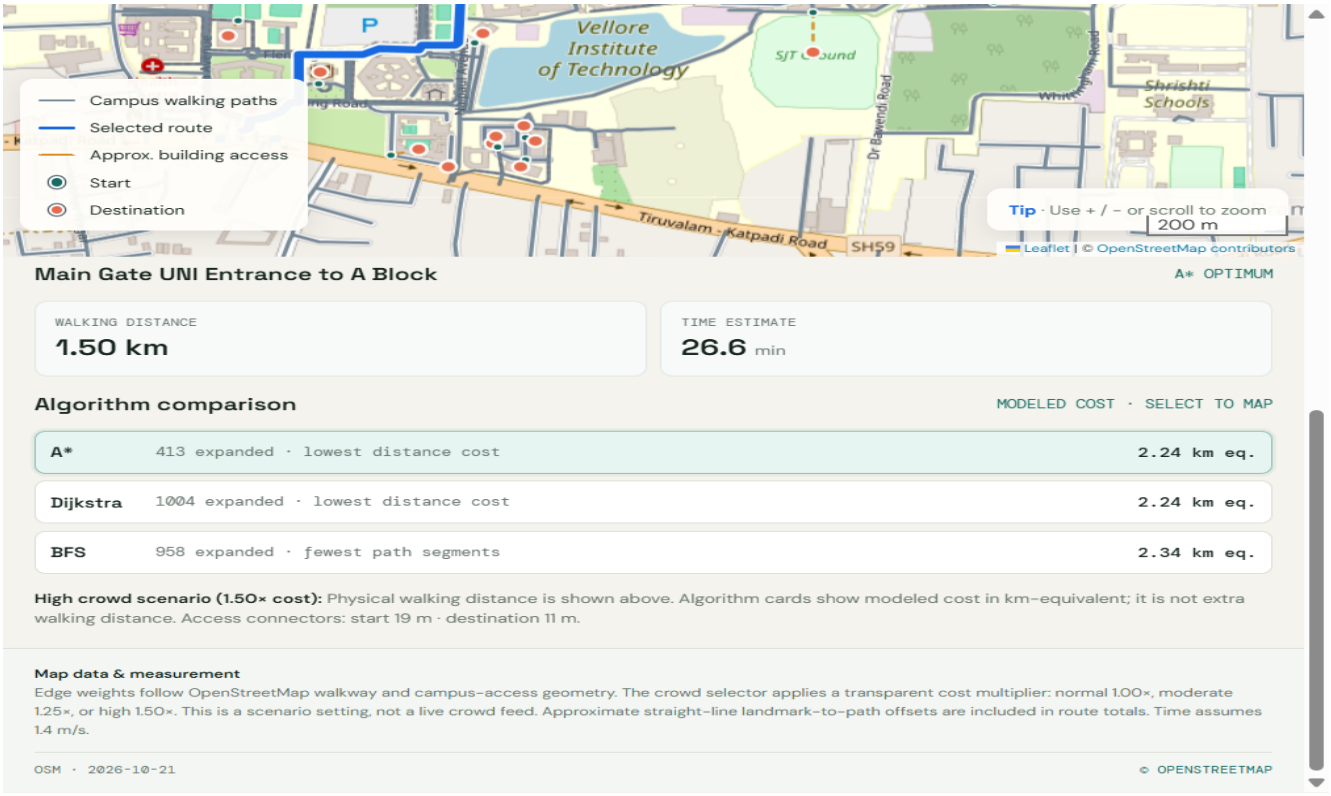


Figure 1. Live browser results: Main Gate UNI Entrance to A Block under the high crowd scenario.

9. Experiments and Results

The experiment was run using the included campus_map.json and the command python campus_nav.py --start main_gate_uni_entrance --goal a_block. Search time is a single-run measurement and is sensitive to machine load. The table distinguishes physical distance from crowd-adjusted modeled cost; modeled cost is reported in m-equivalent and is not additional walking distance.

Algorithm	Physical distance	Modeled cost	Estimated time	Nodes expanded
A*	1,501.9 m	1,501.9 m-eq.	17.9 min	413
Dijkstra	1,501.9 m	1,501.9 m-eq.	17.9 min	1,004
BFS	1,570.5 m	1,570.5 m-eq.	18.7 min	958

A* matches Dijkstra's optimal distance while expanding 58.9% fewer nodes (413 versus 1,004). BFS expands fewer nodes than Dijkstra in this run, but its fewest-hop objective produces a route 68.6 m longer than the optimal weighted route.

Under the high crowd scenario (1.50x), the same A* route has a modeled search cost of 2,238.2 m-equivalent and an estimated time of 26.6 minutes, while its measured physical distance remains 1,501.9 m. This confirms that the crowd feature changes route cost and time without misrepresenting physical map distance.

Sample input: `python campus_nav.py --start main_gate_uni_entrance --goal a_block`
Sample output: A* returns a 1,501.9 m route in 17.9 minutes with 413 expanded nodes. Dijkstra returns the same distance with 1,004 expanded nodes; BFS returns 1,570.5 m with 958 expanded nodes.

10. Validation and Testing

The automated test suite contains seven tests and all passed. It checks A* against Dijkstra on the VIT graph; verifies BFS's different weighted-distance behavior; handles start-equals-goal and unreachable cases; reports unknown locations; ensures hostel display names are unique; and rejects an edge that would violate heuristic admissibility.

11. Discussion, Limitations and Possible Improvements

• OSM is community-maintained and is not an official VIT navigation dataset.

• Named buildings may not have mapped pedestrian entrances; endpoint snaps are estimates.

• The current graph treats edges as undirected and does not model live closures.

• Walking time assumes a constant 1.4 m/s and is not a live prediction.

• Live crowd sensing, step-free accessibility, terrain difficulty, and personal mobility are not currently modeled. The crowd selector is a user-selected cost scenario.

• Map tiles and data are attributed to OpenStreetMap contributors under the ODbL.

12. Team Contributions and References

Member	Contribution
Rohit Arya	Mapped the campus coordinate graph, defined weighted edges, and implemented baseline Dijkstra/BFS comparison algorithms.
Nayan Gattani	Implemented the A* search engine, admissible Euclidean heuristic, path output, and validation logic.
Aditya Kumar	Integrated the local web service and browser demonstration workflow, including route API handling and presentation of results.
Ashish Kumar	Supported testing, experimental comparison, documentation, and live demonstration preparation.

13. Conclusion

The project demonstrates how informed graph search can solve a practical campus navigation problem. A* produces the same minimum-distance route as Dijkstra while using the Euclidean heuristic to reduce exploration. BFS provides a useful contrast: fewest hops do not necessarily mean shortest walking distance. The working CLI, browser application, automated tests, and live route comparison satisfy the project objectives and provide a clear basis for the final demonstration and viva voce.

References

1. Campus_Navigation_Proposal (2).pdf, submitted project proposal.

2. OpenStreetMap contributors, campus walkway data and map tiles, accessed through the project dataset.

3. Project source files: campus_nav.py, web_app.py, index.html, campus_map.json, README.md, and test_campus_nav.py.

Source-code and demonstration access: The complete source code and test data are in the submitted project workspace. During evaluation, run `python -m unittest -v` for automated tests, `python campus_nav.py` for the CLI, or `python web_app.py` and open `http://127.0.0.1:8000` for the live demonstration.

Generative-AI assistance disclosure: Generative AI was used as an editing and documentation assistant to organize this report and presentation, improve wording, and review test evidence. The team implemented, ran, tested, and verified the AI search algorithms and remains responsible for the code, results, interpretations, and viva explanations.